

Competitive Programming

Longest Common Subsequence

and

Knapsack

Longest Common Subsequence

- Consider a sequence: 5, 9, 3, 2, 1, 6, 4
- A subsequence, take some items in order
 - Like: 5, 3, 1, 4
 - Or: 9, 1, 6
- Given two subsequences, find the longest sequence that's a subsequence of both:
 - 5, 9, 3, 2, 1, 6, 4
 - 3, 9, 6, 2, 4, 1, 8

Longest Common Subsequence

- Consider a sequence: 5, 9, 3, 2, 1, 6, 4
- A subsequence, take some items in order
 - Like: 5, 3, 1, 4
 - Or: 9, 1, 6
- Given two subsequences, find the longest sequence that's a subsequence of both:
 - 5, 9, 3, 2, 1, 6, 4
 - 3, 9, 6, 2, 4, 1, 8
 - It's 3, 6, 4

Longest Common Subsequence

- Consider a sequence: 5, 9, 3, 2, 1, 6, 4
- A subsequence, take some items in order
 - Like: 5, 3, 1, 4
 - Or: 9, 1, 6
- Given two subsequences, find the longest sequence that's a subsequence of both:
 - 5, 9, 3, 2, 1, 6, 4
 - 3, 9, 6, 2, 4, 1, 8
 - It's 3, 6, 4
 - Or 9, 2, 1

Length of the common subsequence is unique.

Could be multiple, equally long subsequences.

Longest Common Subsequence via DP

- Consider two strings:

- abacba
- bacbaba

Longest subsequence of a
prefix of each string.

Longest subsequence of
abac and bacba

	b	a	c	b	a	b	a
a							
b							
a							
c							
b							
a							

Longest Common Subsequence via DP

- Consider two strings:

- abacba
- bacbaba

Assuming
characters are
indexed from 1.

	b	a	c	b	a	b	a
a							
b							
a							
c							
b							
a							

Here's your
answer.

Longest Common Subsequence via DP

- Consider two strings:

- abacba
- bacbaba

Trivial base cases.

		b	a	c	b	a	b	a
		0	0	0	0	0	0	0
a		0						
b		0						
a		0						
c		0						
b		0						
a		0						

Longest Common Subsequence via DP

- Consider two strings:
 - abacba
 - bacbaba

		b	a	c	b	a	b	a
a	0	0	0	0	0	0	0	0
b	0	0						
a	0							
c	0							
b	0							
a	0							

Longest Common Subsequence via DP

- Consider two strings:
 - abacba
 - bacbaba

		b	a	c	b	a	b	a
a	0	0	0	0	0	0	0	0
b	0							
a	0							
c	0							
b	0							
a	0							

Longest Common Subsequence via DP

- Consider two strings:
 - abacba
 - bacbaba

		b	a	c	b	a	b	a
a	0	0	0	0	0	0	0	0
b	0							
a	0							
c	0							
b	0							
a	0							

Longest Common Subsequence via DP

- Consider two strings:

- abacba
- bacbaba

		b	a	c	b	a	b	a
a	0	0	0	0	0	0	0	0
b	0	1						
a	0							
c	0							
b	0							
a	0							

Longest Common Subsequence via DP

- Consider two strings:

- abacba
- bacbaba

		b	a	c	b	a	b	a
		0	0	0	0	0	0	0
a		0	0	1	1	1	1	1
b		0	1	1	1			
a		0						
c		0						
b		0						
a		0						

Longest Common Subsequence via DP

- Consider two strings:
 - abacba
 - bacbaba

		b	a	c	b	a	b	a
		0	0	0	0	0	0	0
a		0	0	1	1	1	1	1
b		0	1	1	1	2		
a		0						
c		0						
b		0						
a		0						



Longest Common Subsequence via DP

- Consider two strings:
 - abacba
 - bacbaba

		b	a	c	b	a	b	a
a	0	0	0	0	0	0	0	0
b	0	1	1	1	2	2	2	2
a	0							
c	0							
b	0							
a	0							

Longest Common Subsequence via DP

- Consider two strings:

- abacba
- bacbaba

		b	a	c	b	a	b	a
		0	0	0	0	0	0	0
a		0	0	1	1	1	1	1
b		0	1	1	1	2	2	2
a		0	1	2				
c		0						
b		0						
a		0						



Longest Common Subsequence via DP

- Consider two strings:
 - abacba
 - bacbaba

		b	a	c	b	a	b	a
		0	0	0	0	0	0	0
a		0	0	1	1	1	1	1
b		0	1	1	1	2	2	2
a		0	1	2	2	2	3	
c		0						
b		0						
a		0						



Longest Common Subsequence via DP

- Consider two strings:

- abacba
- bacbaba

		b	a	c	b	a	b	a
a		0	0	0	0	0	0	0
b		0	1	1	1	2	2	2
a		0	1	2	2	2	3	3
c		0	1	2	3	3	3	3
b		0	1	2	3	4	4	4
a		0	1	2	3	4	5	5

Longest Common Subsequence via DP

- $T[i][j]$ = length of LCS of $a_1 \dots a_i$ and $b_1 \dots b_j$.
- $T[i][j] = \max(\begin{array}{l} T[i-1][j], \\ T[i][j-1], \\ T[i-1][j-1] + 1 \text{ if } a_i == b_j \end{array})$

Recovering a Longest Subsequence

- We can recover a longest subsequence by
 - Starting at the bottom-right corner.
 - Working our way back to a base case.
 - Following an explanation for each value.
- Example: **a**

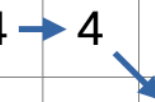
		b	a	c	b	a	b	a
		0	0	0	0	0	0	0
a		0	0	1	1	1	1	1
b		0	1	1	1	2	2	2
a		0	1	2	2	2	3	3
c		0	1	2	3	3	3	3
b		0	1	2	3	4	4	4
a		0	1	2	3	4	5	5



Recovering a Longest Subsequence

- We can recover a longest subsequence by
 - Starting at the bottom-right corner.
 - Working our way back to a base case.
 - Following an explanation for each value.
- Example: **a**

		b	a	c	b	a	b	a
		0	0	0	0	0	0	0
a		0	0	1	1	1	1	1
b		0	1	1	1	2	2	2
a		0	1	2	2	2	3	3
c		0	1	2	3	3	3	3
b		0	1	2	3	4	4	4
a		0	1	2	3	4	5	5



Recovering a Longest Subsequence

- We can recover a longest subsequence by
 - Starting at the bottom-right corner.
 - Working our way back to a base case.
 - Following an explanation for each value.
- Example: **ba**

		b	a	c	b	a	b	a
		0	0	0	0	0	0	0
a		0	0	1	1	1	1	1
b		0	1	1	1	2	2	2
a		0	1	2	2	2	3	3
c		0	1	2	3	3	3	3
b		0	1	2	3	4	4	4
a		0	1	2	3	4	5	5

Recovering a Longest Subsequence

- We can recover a longest subsequence by
 - Starting at the bottom-right corner.
 - Working our way back to a base case.
 - Following an explanation for each value.
- Example: **bacba**

		b	a	c	b	a	b	a
		0	0	0	0	0	0	0
a	0	0	1	1	1	1	1	1
b	0	1	1	1	2	2	2	2
a	0	1	2	2	2	3	3	3
c	0	1	2	3	3	3	3	3
b	0	1	2	3	4	4	4	4
a	0	1	2	3	4	5	5	5

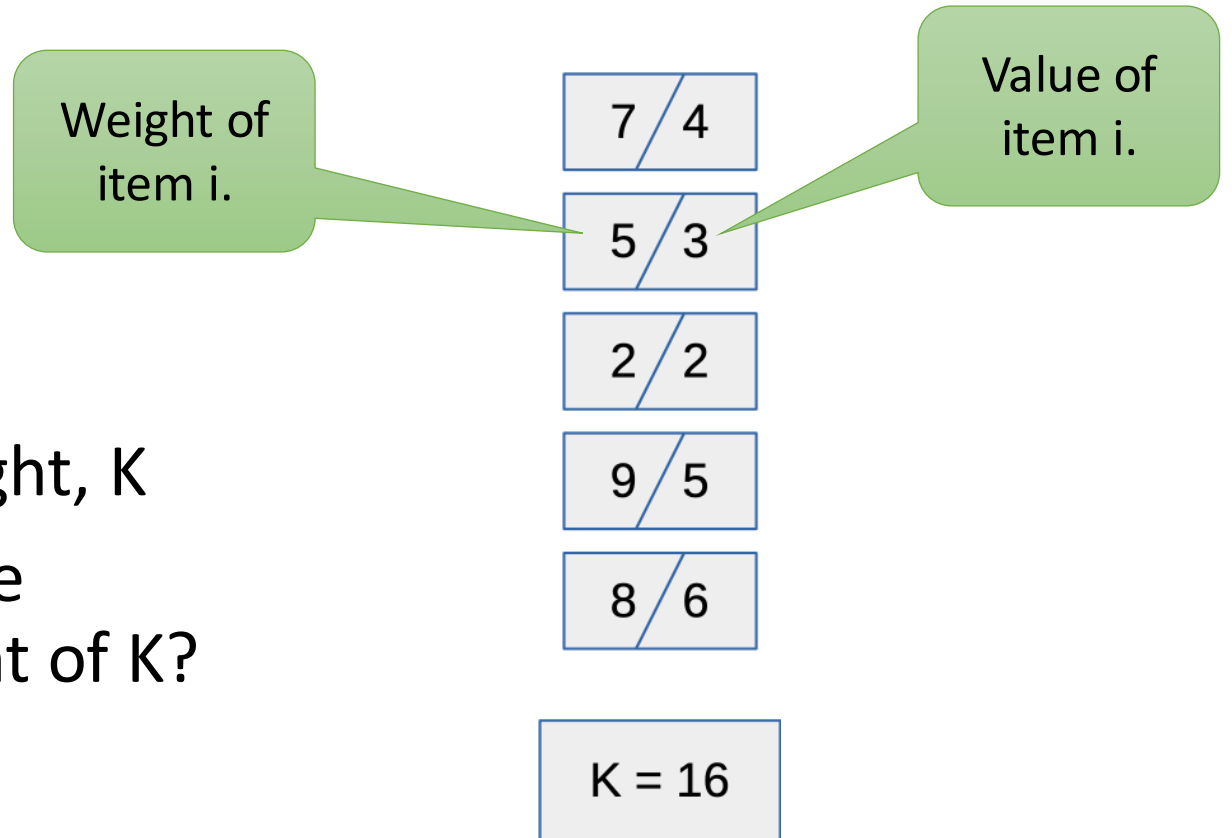
Recovering a Longest Subsequence

- There can be choices when recovering a longest subsequence.
- In general, this can give alternative solutions.
- ... but not here.
- Another example: **bacba**

		b	a	c	b	a	b	a
		0	0	0	0	0	0	0
a	0	0	1	1	1	1	1	1
b	0	1	1	1	2	2	2	2
a	0	1	2	2	2	3	3	3
c	0	1	2	3	3	3	3	3
b	0	1	2	3	4	4	4	4
a	0	1	2	3	4	5	5	5

Knapsack

- Given a set of n items
 - Each with a weight: $w_1 \dots w_n$
 - And a value: $v_1 \dots v_n$
- Given a bound on the total weight, K
- What's the maximum total value without exceeding a total weight of K ?



Knapsack

- Given a set of n items
 - Each with a weight: $w_1 \dots w_n$
 - And a value: $v_1 \dots v_n$
- Given a bound on the total weight, K
- What's the maximum total value without exceeding a total weight of K ?

7 / 4

5 / 3

2 / 2

9 / 5

8 / 6

16 / 9

Knapsack

- Given a set of n items
 - Each with a weight: $w_1 \dots w_n$
 - And a value: $v_1 \dots v_n$
- Given a bound on the total weight, K
- What's the maximum total value without exceeding a total weight of K ?

7 / 4

5 / 3

2 / 2

9 / 5

8 / 6

16 / 10

Knapsack

- Given a set of n items
 - Each with a weight: $w_1 \dots w_n$
 - And a value: $v_1 \dots v_n$
- Given a bound on the total weight, K
- What's the maximum total value without exceeding a total weight of K ?
- We could enumerate subsets of the items, but that only works for small n .

7 / 4

5 / 3

2 / 2

9 / 5

8 / 6

15 / 11

Knapsack in Pseudo-Polynomial Time

- A table with
 - A row for every item.
 - A column for every possible sum up to K .

w	v	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
7	4																	
5	3																	
2	2																	
9	5																	
8	6																	

Best value possible with a total weight of 7 and using just the first two items.

Knapsack in Pseudo-Polynomial Time

- A table with
 - A row for every item.
 - A column for every possible sum up to K .

w	v	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
	0																	
7	4																	
5	3																	
2	2																	
9	5																	
8	6																	

An easy base case.

No items available, so a total weight of zero and a total value of zero.

Here's the answer.

Knapsack in Pseudo-Polynomial Time

- Using just item 1.

<i>w</i>	<i>v</i>	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
	0	0																
7	4	0							4									
5	3																	
2	2																	
9	5																	
8	6																	

Assuming I can have empty cells, for total capacities I can't hit exactly.

Knapsack in Pseudo-Polynomial Time

- Using item 1 and / or 2
 - I can still get any total value possible without item 2.

[illegible]

Knapsack in Pseudo-Polynomial Time

- Using item 1 and / or 2
 - I can still get any total value possible without item 2.
 - Plus any previous solution plus the value of v_2 , with added weight of w_2

[illegible]

Knapsack in Pseudo-Polynomial Time

- Using item 1 and / or 2
 - I can still get any total value possible without item 2.
 - Plus any previous solution plus the value of v_2 , with added weight of w_2
 - Even the solution with just item 2 is an example of this.

[illegible]

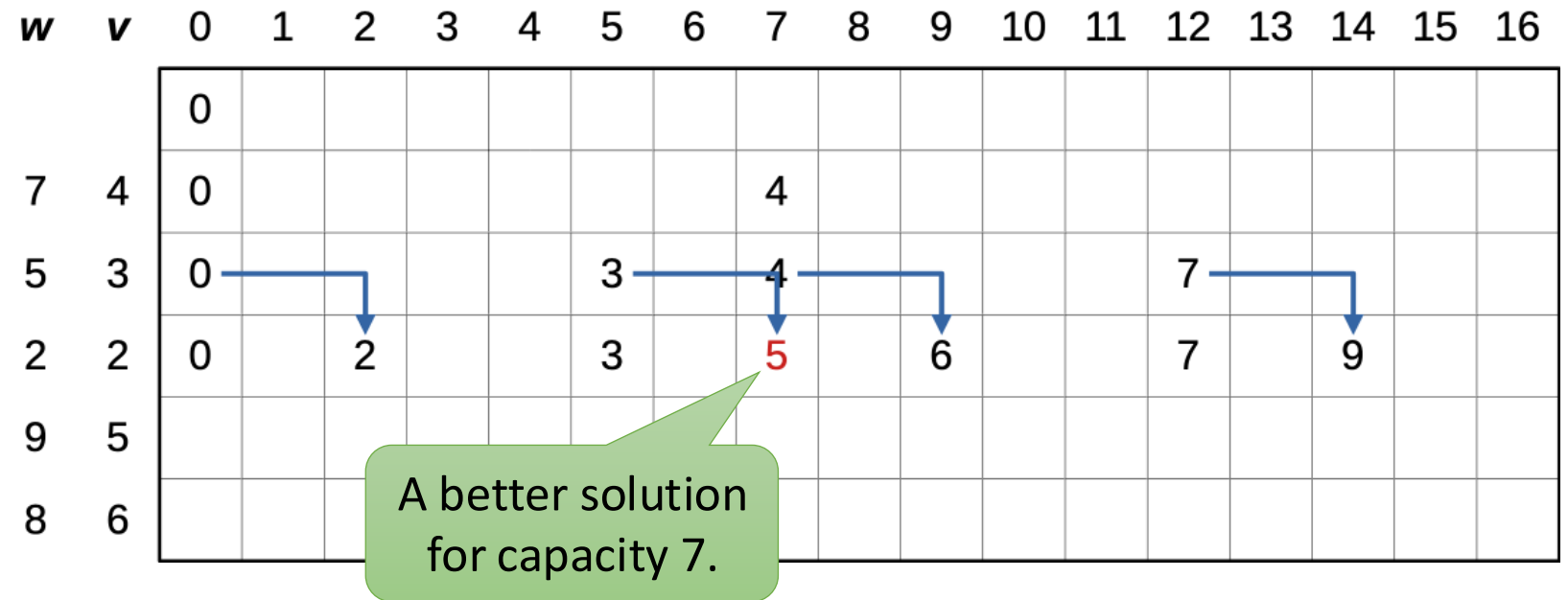
Knapsack in Pseudo-Polynomial Time

- Using any of items 1, 2 and 3.
 - Can still get any total value possible without item 3.

[illegible]

Knapsack in Pseudo-Polynomial Time

- Using any of items 1, 2 and 3.
 - Can still get any total value possible without item 3.
 - But item 3 may give new or better solutions.
 - Take the maximum of $T[i-1][j]$ and $T[i-1][j-w_3] + v_3$



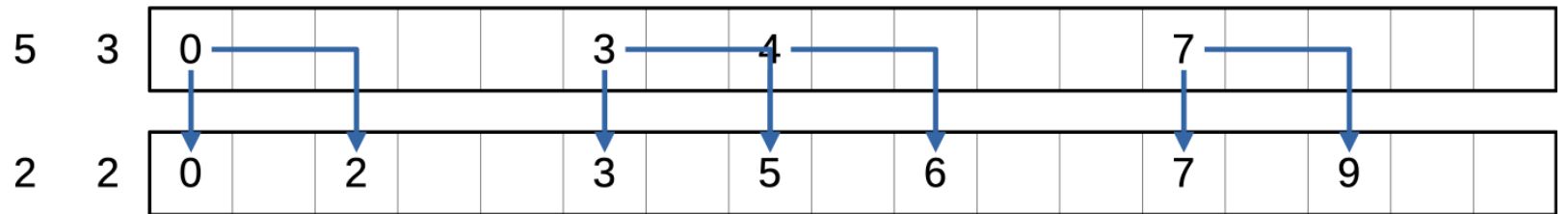
Knapsack in Pseudo-Polynomial Time

- Using any of items 1 .. 5
 - Bottom row has the best solutions, for any capacity, using any subset of the items.
 - Maximum value may not need all K capacity.

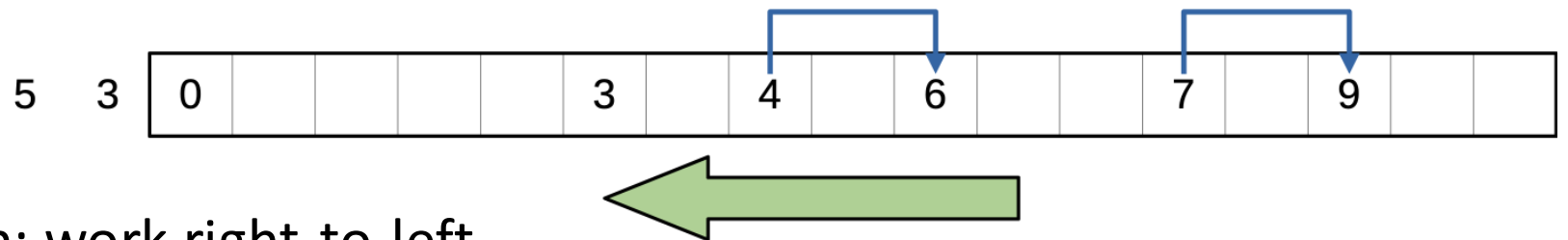
<i>w</i>	<i>v</i>	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
	0																	
7	4	0							4									
5	3	0					3		4					7				
2	2	0		2			3		5		6			7		9		
9	5	0		2			3		5		6		7	7		9		10
8	6	0		2			3		5	6	6	8	7	7	9	9	11	10

Knapsack in Pseudo-Polynomial Time

- This is a serial DP algorithm
 - You don't need to store all the rows.



- You can even update a row in place, but don't update a value using a value you



- A common solution: work right-to-left.